KV Cache 横纵分析报告

> 它没有一篇"提出论文"、没有首位作者、没有任何学术殿堂的加冕——却在六年里从 HuggingFace 代码库里的一个 `past_key_values` 参数，长成了价值数百亿美元推理基础设施的基石。KV Cache 可能是 AI 工程史上"工程跑在论文前面"最经典的故事。

一、一句话定义

KV Cache（Key-Value Cache）是 Transformer 自回归推理时，将每个 token 投影好的 Key 和 Value 向量缓存下来的优化技术。它让模型每生成一个新 token，不必重新计算所有历史 token 的 K、V 投影，只需从缓存中直接读取。

一句话：**投影一次，存起来，后面直接用。**

> 🎯 **读完这篇你能**：判断一个 LLM 部署场景该选什么 KV Cache 优化方案，理解 MQA/GQA/MLA/PagedAttention 各自解决什么层面的问题、能否叠加，能从技术演进史预判下一个优化方向。

阅读指南

如果你只有 5 分钟：读一句话定义 + 横纵交汇洞察的最后一部分（"三个未来剧本"）。你会在 5 分钟内理解 KV Cache 为什么是推理优化的基石，以及它的未来走向。

如果你想理解 KV Cache 但不碰技术细节：读技术背景的三条线 + 纵向分析的每个阶段开头段和末尾"核心矛盾"表 + 横向分析的开头分类框架和总结。约 25 分钟。

如果你是推理系统开发者/研究者：直接跳横向分析的完整对比表和各方案独立分析。纵向分析的 3.1（诞生：2019 HuggingFace）和 3.5（MLA：范式级突破）是两个必读的关键历史节点。

二、技术背景

> 前置知识请参考独立文档 《KV Cache 前置知识》，涵盖 KV Cache 基本原理、四条优化路线的直观讲解和每个方向的代表方法。

KV Cache 的根本问题可以用一个公式说清。一个 token 的 KV Cache 大小：

```
2 × 层数 × 头数 × 每头维度 × 精度字节数
```

以 Llama 2 7B（32 层、32 头、128 维度、FP16）为例——一个 token 约 0.5MB，一千 token 约 0.5GB。这只是单个请求。生产环境 100 并发、每个请求 4000 token，仅 KV Cache 就吃掉上百 GB 显存。

KV Cache 面临的核心矛盾是：

```
保留所有 token 的 K、V → 显存放不下
丢弃 / 压缩部分 KV → 可能丢失关键信息
```

所有优化方案本质上都在这条钢索上走。四个切入角度——减少 KV 头数（架构层）、分页管理（系统层）、挑选重要 token（序列维）、减少比特数（数值维）——分别从不同的技术层面解决问题。它们可以叠加使用，实际生产部署中几乎都是组合方案。

> 📚 **关联报告**
>
> – LLM 大模型 — KV Cache 是 LLM 推理中最核心的显存消耗源，DeepSeek-V2 的 MLA 是架构层面最重要的突破之一
>
> – 大模型推理部署 — KV Cache 是推理部署中最大的显存瓶颈，PagedAttention、量化等都是推理框架的核心优化
>
> – Transformer — KV Cache 的技术根基是 Transformer 的 Attention 机制，所有优化都是在其上的改进

三、纵向分析：从工程 hack 到学术显学

3.1 概念前史：Transformer-XL 埋下的伏笔（2019.01）

KV Cache 的思想源头可以追溯到 2019 年 1 月的 **Transformer-XL**（ACL 2019）。这篇论文当时的研究动机跟推理优化无关——它要解决的是原版 Transformer"固定长度分段"导致的上下文断裂问题。

Transformer-XL 的做法是：处理新一段时，把上一段每一层的 hidden states 缓存下来，作为扩展上下文参与注意力计算。这个"缓存上一段 K、V 并复用"的模式，在结构上和后来的 KV Cache 高度相似。区别只在于前者缓存的是"上一段"，后者缓存的是"同一序列前面所有已生成的 token"。

Transformer-XL 的引用量和影响让它成为 KV Cache 思想最早的学术源头——虽然论文本身没有讨论推理优化。

3.2 诞生：HuggingFace GPT-2 的一个参数（2019 年中）

KV Cache 的出身毫不光鲜。它不是在某个峰会上被重点发布的，而是从 HuggingFace 的社区 Issues 和 PRs 里生长出来的。

2019 年 2 月 GPT-2 发布后，HuggingFace Transformers 库迅速集成了它。社区很快发现了一个要命的问题：自回归生成时，每个新 token 都要对整个序列重做 Self-Attention——$O(n^2)$ 的计算量随着序列增长变得无法忍受。

解决方案直接到了代码里：HuggingFace 给 GPT-2 模型加了一个叫 `past_key_values`（早期叫 `past` / `layer_past`）的参数，用于缓存之前 token 各层的 K、V 投影结果。

Issue #2368（2019 年 12 月）有用户询问 `past` 参数的用法。**PR #9596**（2020 年 6 月）提交了改进的缓存格式统一方案。**Issue #9391**（2020 年 6 月）讨论在 CausalLM 和 Seq2SeqLM 之间统一 `past_key_values` 的用法。

这就是 KV Cache 的诞生。没有论文、没有第一作者、没有白皮书——只有社区对"怎么让它跑得更快"的朴素追求。但这个"工程 hack"的质量如此之高，以至于此后几乎每一篇关于 Transformer 推理优化的论文都必须讨论 KV Cache。

3.3 第一个架构级优化：MQA 与 GQA（2019-2023）

MQA（2019.11） — 虽然 KV Cache 本身没有"提出论文"，但 KV Cache 优化的第一篇重要论文是同一年发布的。Noam Shazeer（Transformer 八位作者之一）在《Fast Transformer Decoding: One Write-Head is All You Need》中精确指出了 KV Cache 的瓶颈：自回归解码时，必须反复加载所有 head 的 K 和 V 大张量。他提出的解决方案极其激进——所有注意力头共享同一套 K、V。KV Cache 减少因子等于注意力头数（H 倍）。

Google 在 PaLM（540B）、GLaM（1.2T）中采用了 MQA。但质量损失在学术圈有争议。

GQA（2023.05） — Google 的 Joshua Ainslie 等人提出了折中方案：将 Q 头分为 G 组，每组共享 KV。这是 MHA（G=H）和 MQA（G=1）之间的平滑插值。更重要的是，论文证明了从已有的 MHA checkpoint 通过"平均池化合并"做少量微调（约 5% 的原始预训练计算量）即可转换到 GQA。

GQA 的质量-效率平衡点找得恰到好处。到 2026 年，Llama 2/3/4、Mistral 全系、Qwen 2/2.5/3、Gemma、Phi-3/4 等几乎所有主流开源模型都默认使用 GQA。它已经成为事实上的行业标准。

3.4 系统革命：PagedAttention 与分页管理（2023）

2023 年下半年，KV Cache 优化的焦点从"单个 token 的 KV 多大"转向"多个用户的 KV 怎么存"。

传统的 KV Cache 管理方式是给每个请求预分配**连续**显存块。问题出在"连续"上——序列长度不可预测，预先分配的连续块不是太大就是太小，导致内碎片和外碎片严重。实际系统内存利用率只有 20-40%。

UC Berkeley 的 **PagedAttention**（vLLM，SOSP 2023）把操作系统的虚拟内存分页搬进 GPU：KV Cache 切成固定大小的 block，按需动态分配，不需要连续存储。在注意力计算时通过 block table 重映射实现逻辑连续访问。

效果直接：内存浪费从 60-80% 降到 4%，吞吐量比 HuggingFace Transformers 快 2-4x，比 NVIDIA FasterTransformer 快达 22x。而且**零精度损失**——纯系统层优化，完全不动模型。

vLLM 论文成为 SOSP 近十年最高引论文之一，vLLM 仓库到 2026 年已有约 57k stars。AWS Bedrock、Databricks、Red Hat、Cloudflare 等将其作为推理后端。

同期，LMSYS 的 **SGLang / RadixAttention**（2023.12）用基数树管理 KV Cache 前缀共享——多个请求的相同前缀（如同一 system prompt）在树中自动合并，避免重复缓存。这和 PagedAttention 不是竞争，而是互补：PagedAttention 管理"页"，RadixAttention 管理"前缀语义"。

3.5 序列维与数值维的并行探索（2023-2024）

2023-2024 年是 KV Cache 优化的爆发期，两个新方向几乎同时打开。

序列维——不是所有 token 都值得缓存。 StreamingLLM（2023.09，MIT/Meta）发现了 Attention Sink 现象：前 4 个 token 吸引了不成比例的高注意力分数，即使语义上完全无关。保留它们 + 最近窗口，中间的丢弃——KV Cache 大小恒定，可以无限长度流式生成。H2O（2023.06，Rice）发现约 20% 的 token 贡献 80% 注意力分数（Heavy Hitters），保留这些重磅选手 + 近期 token。SnapKV（2024.04）更聪明——生成前用 prompt 尾部 token 的注意力模式一次性投票决定保留哪些 KV，生成阶段零额外开销。

但这一方向面临一个根本性质疑：NeurIPS 2025 的一篇独立评估论文《Hold Onto That Thought》发现，token 淘汰方法在数学推理和代码生成等复杂推理任务上会导致显著的精度退化。这意味着这些方法对 reasoning model（DeepSeek-R1、o1/o3）可能不够安全。**生产环境的态度普遍保守——宁可多花显存，也不在推理质量上冒险。**

数值维——能不能少存几个比特。 KVQuant（2024.01，UC Berkeley）实现了近乎无损的 4-bit KV 量化，单卡 A100 可处理 100 万 token 上下文。KIVI（2024.02）更进一步——2-bit，免调优。这两个方向同时爆发的原因是：量化不需要改模型架构，不需要重新训练，部署成本趋近于零。到 2026 年，FP8 KV Cache 已经是 vLLM 的标准选项，INT8 KV Cache 被 TensorRT-LLM 原生支持。

3.6 MLA：解决 KV Cache 问题的根本方法?（2024.05）

2024 年 5 月，DeepSeek-V2 技术报告中的一个架构创新让整个推理优化社区震撼。

Multi-head Latent Attention（MLA） 走了一条和 MQA/GQA 完全不同的路。MQA/GQA 在注意力头的数量上做文章——多少套 Q 共享一套 KV。MLA 换了一个问题：**能不能不存 K 和 V，而是存一个低维压缩表示?**

做法是：K 和 V 先投影到一个小的潜在空间，缓存这个压缩后的潜在向量。注意力计算时，通过上投影矩阵恢复等效结果。更精妙的是——通过"矩阵吸收"技术，将上投影矩阵融合进 Q 的投影矩阵中，解码时根本不需要显式重建完整 K、V。

效果：KV Cache 减少约 93%，吞吐量提升 5.76 倍，且**在基准测试上无明显精度损失**。

MLA 的范式意义怎么强调都不过分。前面 5 年的优化都默认"KV 必须显式缓存"这一前提——MQA/GQA 减少头数、量化减少比特数、淘汰减少 token 数。MLA 直接把这前提拆了：**可以不存 K、V，存别的东西。** 这个思路一旦成立，整个优化空间就重新打开了。

2025 年 NeurIPS Spotlight 论文 TransMLA 进一步证明可以从 GQA 迁移到 MLA架构——降低了 MLA 的模型迁移成本。FlashMLA（DeepSeek 开源的 CUDA kernel）在 H800 上达到 3000 GB/s 显存带宽。

3.7 深度维与系统级优化（2024-2026）

MLA 之后，KV Cache 的优化进入了一个新阶段。两个新方向浮现：

跨层共享（深度维）：前面所有方案都在优化"每层 per-token 的 KV"，但 32-80 层的 LLM 中，相邻层的 KV 高度相似。Layer-Condensed KV（2024.05，ACL 2024）发现只需在少数层缓存 KV 即可，吞吐最高提 26x。MiniCache（2024.05，NeurIPS 2024）合并相邻层相似 KV，5x 压缩比。KVSharer（2024.10）做出了反直觉的发现——**不相似的层共享 KV 效果更好**（因为避免了信息冗余）。SimLayerKV（2024.10）的优雅程度令人印象深刻——只需 7 行代码识别出可以跳过 KV 计算的"懒惰层"。

系统级共享（2025-2026）：KV Cache 优化从单请求走向跨请求。KVShare（2025.03）实现多租户 KV Cache 复用。DroidSpeak（NSDI 2026）做到不同 LLM 之间共享 KV Cache——这在此前几乎是不可想象的。"Prefill-as-a-Service"（2026.04）将 KV Cache 做成独立服务。KV Marketplace（2025）甚至提出了跨 GPU 的 KV Cache 交易机制——KV Cache 正在变成一个"可流通的商品"。

3.8 阶段总结

阶段	时间	核心矛盾	标志性工作	解决方式
概念前史	2019.01	段间上下文断裂	Transformer-XL	缓存上一段 hidden states
工程诞生	2019 中	自回归重复计算	HF GPT-2 `past_key_values`	缓存已算好的 K、V
架构1: 减头	2019-2023	KV head 数量 vs 质量	MQA → GQA	减少/分组 KV head
系统革命	2023	连续分配内存碎片	PagedAttention, RadixAttention	分页 + 树形前缀共享
序列+数值并行	2023-2024	长序列 KV 线性膨胀	H2O, SnapKV, KVQuant, KIVI	选择性保留 + 少比特存
范式突破	2024.05	能否不存完整 K、V	MLA (DeepSeek-V2)	低秩潜在空间压缩
深度+系统化	2024-2026	层间冗余, 请求间重复	Layer-Condensed, MiniCache, KVShare	跨层共享 + 跨请求复用

四、横向分析：2026 年 KV Cache 优化竞争图谱

4.1 场景判断

KV Cache 优化赛道属于**场景 C（竞品充分）**——四个方向、十余种具体方案，多维度竞争。但各方案之间不是"替代"关系，而是"叠用"关系——实际部署中几乎都是组合方案。

4.2 完整对比表

技术	推出时间	团队	优化层级	KV Cache 缩减	精度损失	需改模型?	框架支持	开源
MQA	2019.11	Shazeer (Google)	架构（减头）	~Hx (32头=32x)	轻微	是（从头训练）	TRT-LLM, vLLM, llama.cpp	✅

GQA	2023.05	Ainslie et al. (Google)	架构（减头）	H/Gx (Llama2=~8x)	几乎无损	是（可微调迁移）	全框架原生	✅
MLA	2024.05	DeepSeek	架构（压缩）	~93% (~14x)	无明显损失	**是（架构级）**	SGLang(原生), vLLM	✅
PagedAttention	2023.09	Kwon et al. (Berkeley)	系统（内存）	消除60–80%浪费	**零**	否	vLLM	✅
RadixAttention	2023.12	LMSYS	系统（前缀）	前缀复用场景显著	**零**	否	SGLang	✅
H2O	2023.10	Zhang et al. (Rice)	序列（淘汰）	~5x (保留20% token)	20%预算轻微	否	vLLM/SGLang部分	✅
SnapKV	2024.04	Li et al. (UIUC)	序列（淘汰）	2–4x	长上下文极小	否	vLLM, SGLang	✅
StreamingLLM	2023.09	Xiao et al. (MIT/Meta)	序列（淘汰）	恒定内存	PPL略升	否	vLLM, llama.cpp	✅
KVQuant	2024.01	Hooper et al. (Berkeley)	数值（量化）	~4x (4–bit)	<0.1 PPL	否	vLLM(实验)	✅
KIVI	2024.02	Liu et al.	数值（量化）	~8x (2–bit)	2–bit下可接受	否	独立实现	✅
Layer–Condensed	2024.05	—	深度（跨层）	吞吐提升26x	可接受	是（训练时改）	实验	✅
MiniCache	2024.05	—	深度（跨层）	5x 压缩比	—	否	实验	✅

KVShare	2025.03	—	系统（跨请求）	多租户复用	**零**（共享部分）	否	研究	✅

4.3 关键发现

GQA 已经是水电煤。 2026 年几乎所有新开源模型都默认 GQA，不用 GQA 反而需要理由。它在标准部署中不需要被"选择"——它就在那。

MLA 是最大变量。 MLA 的 93% KV 缩减是当前公开方案中最激进的，且精度不降。但代价是架构级改造——现有模型没法直接装一个 MLA。这制造了一个有趣的分裂：新模型（DeepSeek 全系、Kimi、部分中国大模型）可以原生支持 MLA，但存量 Llama/Qwen 模型只能继续用 GQA + 其他方案的组合。

系统层优化是最安全的选择。 PagedAttention 和 RadixAttention 零精度损失、不改模型、开箱即用。对生产环境来说，这类方案没有理由不用。

Token 淘汰在推理任务上有风险。 NeurIPS 2025 的独立评估对这条路线是记重拳。在没有解决推理精度退化之前，生产环境对 token 淘汰方案的态度会保持保守。

KV 量化正在从"学术"走向"标配"。 FP8 KV Cache 已是 vLLM 标准选项——这不再是个"要不要量化"的问题，而是"精度的默认水位线在下降"。

4.4 社区口碑

vLLM / PagedAttention: "让 LLM 推理从实验室走向生产的关键技术"——这是社区最一致的判断。也有声音指出 vLLM 的 monolithic 架构在某些灵活部署场景过于笨重。

GQA: 无争议。被视为理所当然的存在。新模型不用 GQA 反而会被社区质疑。

MLA: 技术社区认可度最高，但部署门槛不低——MLA kernel 对 GPU（最好是 H800/H100）有要求。SGLang 的 MLA 优化被视为其核心竞争力。

Token 淘汰: 争议最大。NeurIPS 2025 评估论文之后，社区里"token dropping 不安全"的声音明显盖过了"值得一试"。一条 Hacker News 高赞评论精准概括了这个矛盾："KV cache eviction is incredible engineering that I'll probably never use in production."

五、横纵交汇洞察

5.1 没有提出者的技术，成了最核心的基础设施

KV Cache 是整个 AI 推理栈中最奇怪的基础设施——它没有提出论文、没有首作者、也不需要引用任何一篇论文才能用。它从 HuggingFace 的一个 GitHub Issue 里长出来，靠的是"这太慢了，得想个办法"的工程直觉。

这个出身决定了它的气质：KV Cache 优化的评价标准从来不是"在某个基准上提高了几个点"，而是"一卡能多跑几个用户"。这个务实基因一路延续到 PagedAttention、RadixAttention——它们发表的地方不是 ML 顶会（PagedAttention

发在系统顶会 SOSP），但它们的影响力比大多数 ML 顶会论文大得多。

5.2 四条路，两个阵营

如果把纵向线上所有的优化方案重新排列，会发现它们分属两个根本不同的阵营：

"不改变注意力"的阵营——PagedAttention、RadixAttention、KV 量化、token 淘汰。这些不改模型、不改注意力计算，只在"怎么存"上做文章。优点是无精度损失或损失可控、部署成本低。缺点是每个方向的优化空间有天花板——页分得再好，也不能多出显存来。

"改变注意力"的阵营——MQA、GQA、MLA、跨层共享。这些从架构层面重新设计注意力的形式，在保证表达能力的前提下减少缓存量。优点是优化上限高。缺点是需要改模型架构、需要重新训练或微调。

这两个阵营的关系很像数据库优化里"调参数"和"改 schema"的区别。前者快、安全但天花板有限；后者上限高但门槛也高。实际部署中，前者先用上（PagedAttention + KV 量化），后者等下一代模型（GQA 已是标配，MLA 正在扩散）。

5.3 MLA 的深层启示：打破"必须存 KV"的假设

五年里所有人都在 KV Cache 的前提内优化。MLA 问了一个没人问的问题：**为什么一定要存 K 和 V?**

这个问题的威力在于，一旦"KV 必须显式缓存"这个前提被拆掉，整个设计空间就重新打开了。低秩压缩之外，有没有其他数学变换可以达到类似效果？YOCO 的双解码器架构证明了另一种路线可行。Kimi 的注意力残差走了第三条路。

MLA 之后，KV Cache 优化从"怎么存得更省"变成了"存什么"。这个范式的转换，可能在五年后回头看时，比 MLA 本身的具体实现重要得多。

5.4 三个未来剧本

剧本一（概率 55%）：GQA + PagedAttention + KV 量化成为标准三元组，MLA 在新模型侧扩散

GQA（不改也行，新模型都带着）、PagedAttention（零成本零损失的显存管理）、KV FP8 量化（零训练的数值优化）组成标准部署栈。MLA 在新模型设计中越来越多被采用，但存量 GQA 模型继续用旧栈。推理框架的默认配置就是这三件套。

剧本二（概率 25%）：KV Cache 本身被重新定义——从"模型的本地记忆"变成"共享服务"

KVShare、DroidSpeak 等系统级共享方案成熟。system prompt 的 KV Cache 预计算成标准步骤。大型推理服务商开始对外提供"KV Cache 即服务"——你请求的 prompt 如果跟别人一样，直接复用已有的。KV Cache 从技术细节变成商业产品。

剧本三（概率 20%）：新架构从根本上消灭 KV Cache 问题

如果线性注意力、状态空间模型（Mamba 系）或混合架构在长序列任务上追平甚至超越 Transformer，KV Cache 本身可能就变得不那么重要了。不是被优化了，是被绕过了。MLA 已经证明"不存完整 K、V 也能干活"。下一步可能是"根本不存 K、V"。

六、信息来源

核心论文与文章

论文/文章	来源	访问时间
Transformer-XL (Dai et al.)	arxiv.org/abs/1901.02860	2026-05-03
Fast Transformer Decoding: One Write-Head is All You Need (MQA, Shazeer)	arxiv.org/abs/1911.02150	2026-05-03
GQA: Training Generalized Multi-Query Transformer Models (Ainslie et al.)	arxiv.org/abs/2305.13245	2026-05-03
FlashAttention (Dao et al.)	arxiv.org/abs/2205.14135	2026-05-03
PagedAttention / vLLM (Kwon et al.)	arxiv.org/abs/2309.06180	2026-05-03
SGLang / RadixAttention (Zheng et al.)	arxiv.org/abs/2312.07104	2026-05-03
H2O (Zhang et al.)	arxiv.org/abs/2306.14048	2026-05-03
StreamingLLM (Xiao et al.)	arxiv.org/abs/2309.17453	2026-05-03
SnapKV (Li et al.)	arxiv.org/abs/2404.14469	2026-05-03
DeepSeek-V2 / MLA	arxiv.org/abs/2405.04434	2026-05-03
KVQuant (Hooper et al.)	arxiv.org/abs/2401.18079	2026-05-03
KIVI (Liu et al.)	arxiv.org/abs/2402.02750	2026-05-03
Layer-Condensed KV Cache	arxiv.org/abs/2405.10637	2026-05-03
Cross-Layer Attention	arxiv.org/abs/2405.12981	2026-05-03
MiniCache	arxiv.org/abs/2405.14366	2026-05-03

KV Cache Compression Survey	arxiv.org/abs/2412.19442	2026-05-03
Comparative Characterization of KV Cache (2026)	arxiv.org/abs/2604.05012	2026-05-03
Hold Onto That Thought (NeurIPS 2025)	nips.cc	2026-05-03

产品与技术来源

来源	URL	访问时间
HuggingFace Transformers Issues (#2368, #9391, PR #9596)	github.com/huggingface/transformers	2026-05-03
vLLM GitHub	github.com/vllm-project/vllm	2026-05-03
SGLang GitHub	github.com/sgl-project/sglang	2026-05-03
NVIDIA FasterTransformer	github.com/NVIDIA/FasterTransformer	2026-05-03
FlashMLA GitHub (DeepSeek)	github.com/deepseek-ai/FlashMLA	2026-05-03
Awesome-KV-Cache-Optimization	github.com/jjiantong/Awesome-KV-Cache-Optimization	2026-05-03

本文是横纵分析系列的第 36 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法。